	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 9

Lab Title: **Multplexer and DE Multiplexer**

Student Name: **Muhamamd Waqas** **Reg. No:** **240756**

Objective: **To have a basic understanding of multiplexer an demultiplexer and verify their operations**

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

EXPERIMENT 09

Multiplexer and De-Multiplexer

Objectives:

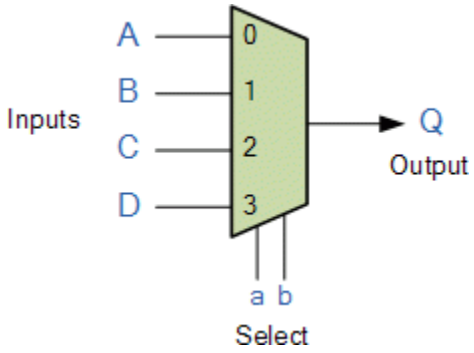
- To have a basic understanding of multiplexers and de-multiplexers.
- To experimentally verify the operation of multiplexers and de-multiplexers.

Equipment required:

- ICs as required
- Digital Electronics Trainer

Background Knowledge:

The multiplexer is a combinational logic circuit designed to switch one of several input lines to a single common output line. It is a combinational logic circuit designed to switch one of several input lines through to a single common output line by the application of a control signal. The block diagram of a multiplexer is shown below:



Multiplexers operate like very fast acting multiple position rotary switches connecting or controlling multiple input lines called "channels" one at a time to the output. Multiplexers, or MUX's, can be either digital circuits made from high speed logic gates used to switch digital or binary data or they can be analogue types using transistors, MOSFET's or relays to switch one of the voltage or current inputs through to a single output. We can show the selection of the data through the multiplexer as a function of the data select bits as shown:

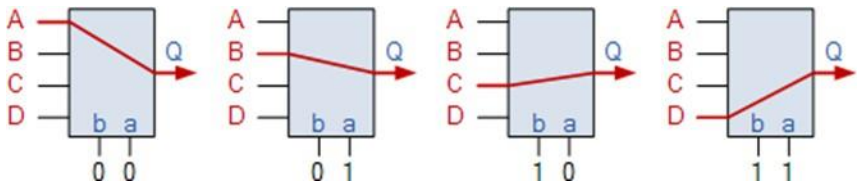
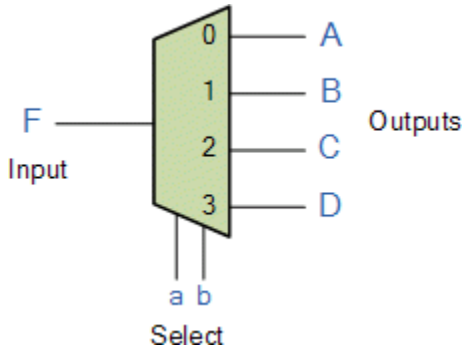


Figure 1: Mux's Input Lines Selection

The demultiplexer is a combinational logic circuit designed to switch one common input line to one of several separate output line. The data distributor, known more commonly as a Demultiplexer or "Demux" for short, is the exact opposite of the Multiplexer. The demultiplexer takes one single input data line and then switches it to any one of a number of individual output lines one at a time. The block diagram of a de-multiplexer is shown below:



The function of the Demultiplexer is to switch one common data input line to any one of the 4 output data lines A to D. As with the multiplexer the individual solid state switches are selected by the binary input address code on the output select pins "a" and "b" as shown.

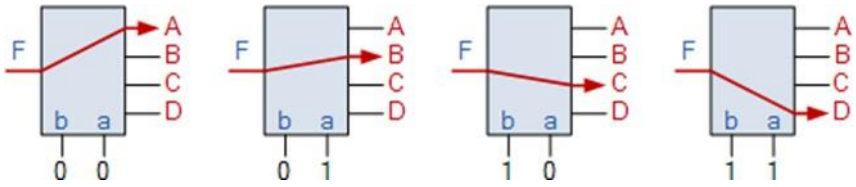


Figure 2: De-Mux's Output Selection Lines

A Multiplexer is used in numerous applications, where multiple data can be transmitted using a single line. Some of them are as follows:

- **Communication System** – A Multiplexer is used in communication systems, which has a transmission system and also a communication network. A Multiplexer is used to increase the efficiency of the communication system by allowing the transmission of data such as audio & video data from different channels via cables and single lines.
- **Computer Memory** – A Multiplexer is used in computer memory to keep up a vast amount of memory in the computers, and also to decrease the number of copper lines necessary to connect the memory to other parts of the computer.
- **Telephone Network** – A multiplexer is used in telephone networks to integrate the multiple audio signals on a single line of transmission.
- **Transmission from the Computer System of a Satellite** – A Multiplexer is used to transmit the data signals from the computer system of a satellite to the ground system by using GSM.

Task 1:

Implement a 4x1 multiplexer. Write complete logic synthesis.

4×1 Multiplexer — Logic Synthesis

Inputs:-

- **Data inputs:**
  I0,I1,I2,I3I_0, I_1, I_2, I_3I0,I1,I2,I3
- **Select lines:**
  S0,S1S_0, S_1S0,S1

=====(Selection Lines Combination)====		
S1	**So**	**Output Y**
0	**0**	**A**
0	**1**	**B**
1	**0**	**C**
1	**1**	**D**

Truth Table:

Input						**Output**
a	**b**	**A**	**B**	**C**	**D**	**Q**
0	**0**	**1**	**0**	**0**	**0**	**A(ON)**
0	**1**	**0**	**1**	**0**	**0**	**B(ON)**
1	**0**	**0**	**0**	**1**	**0**	**C(0N)**
1	**1**	**0**	**0**	**0**	**1**	**D(ON)**

Canonical Sum-of-Products (use S1' to mean NOT S1, S0' to mean NOT S0):

Y = S1' * S0' * I0 + S1' * S0 * I1 + S1 * S0' * I2 + S1 * S0 * I3

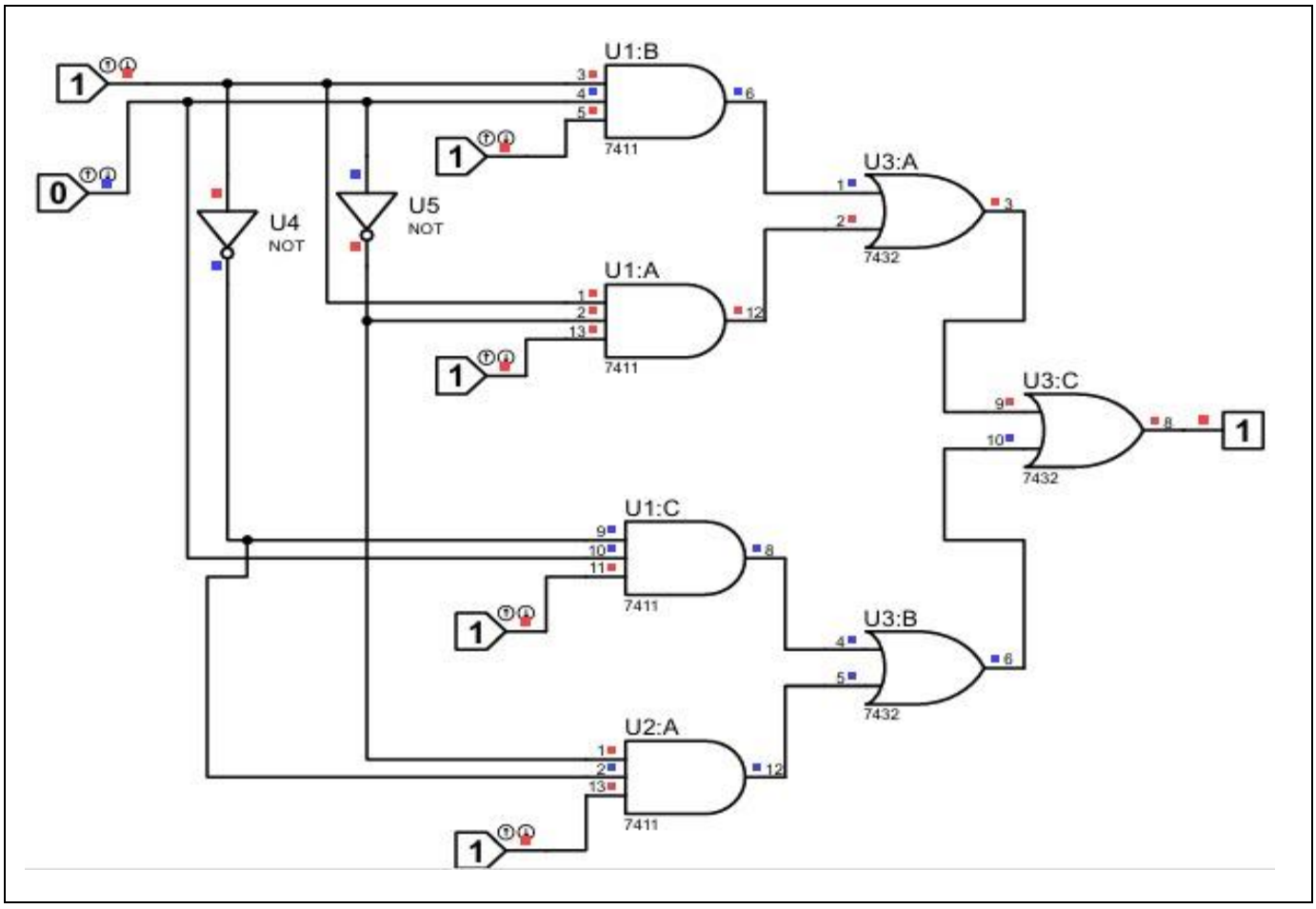
Logic gate-level terms (define intermediate terms T0..T3):
A = S1' * S0' * I0 (active when S1=0, S0=0)
B = S1' * S0 * I1 (active when S1=0, S0=1)
C = S1 * S0' * I2 (active when S1=1, S0=0)
D = S1 * S0 * I3 (active when S1=1, S0=1)

Final output as OR of the 4 terms:
Y = T0 + T1 + T2 + T3

Required logic gates:

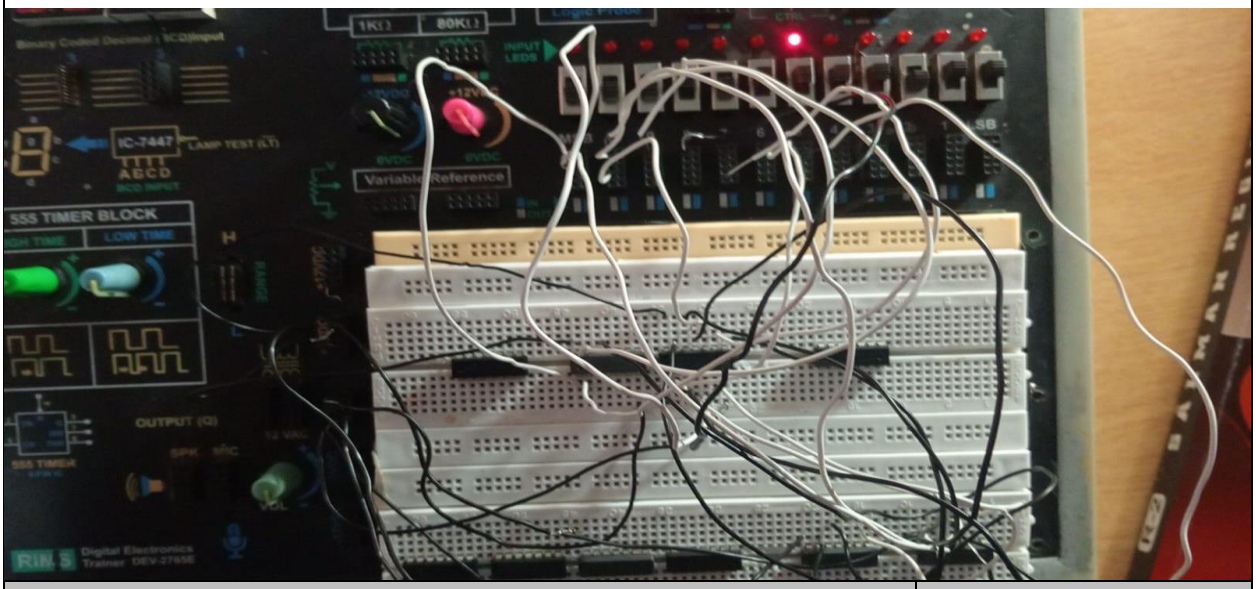
- **NOT gates:**
  - 1 NOT to produce S1' from S1
  - 1 NOT to produce S0' from S0
- **AND gates**:
  - Four 3-input AND gates producing T0, T1, T2, T3
- **OR gate**:
  - One 4-input OR gate to combine T0..T3 into Y

Logic Diagram:

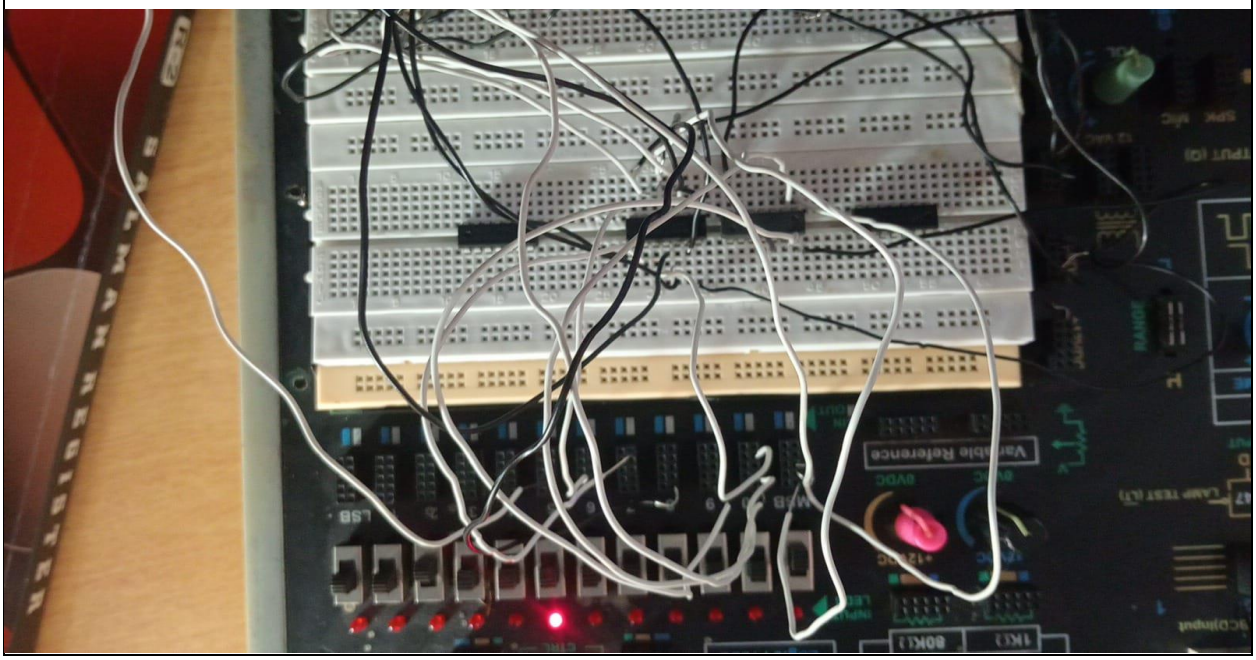


Simulation Activity:

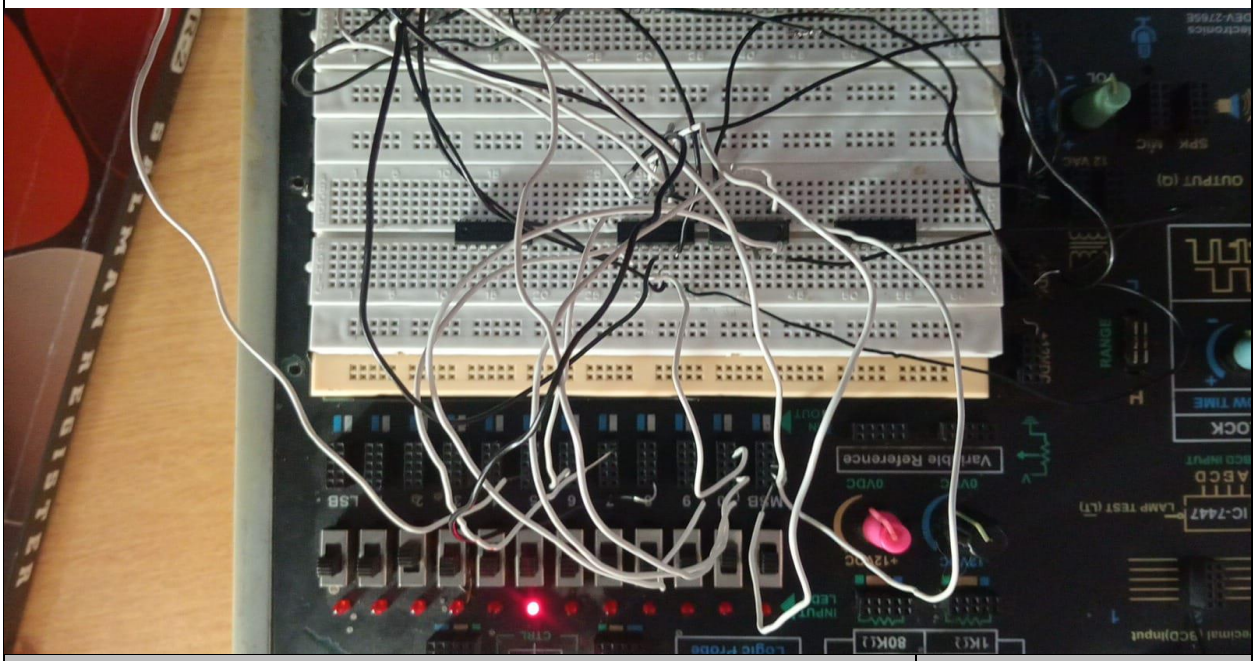
Input						Output
a	**b**	**A**	**B**	**C**	**D**	**Q**
0	**0**	**1**	**0**	**0**	**0**	**A(ON)**



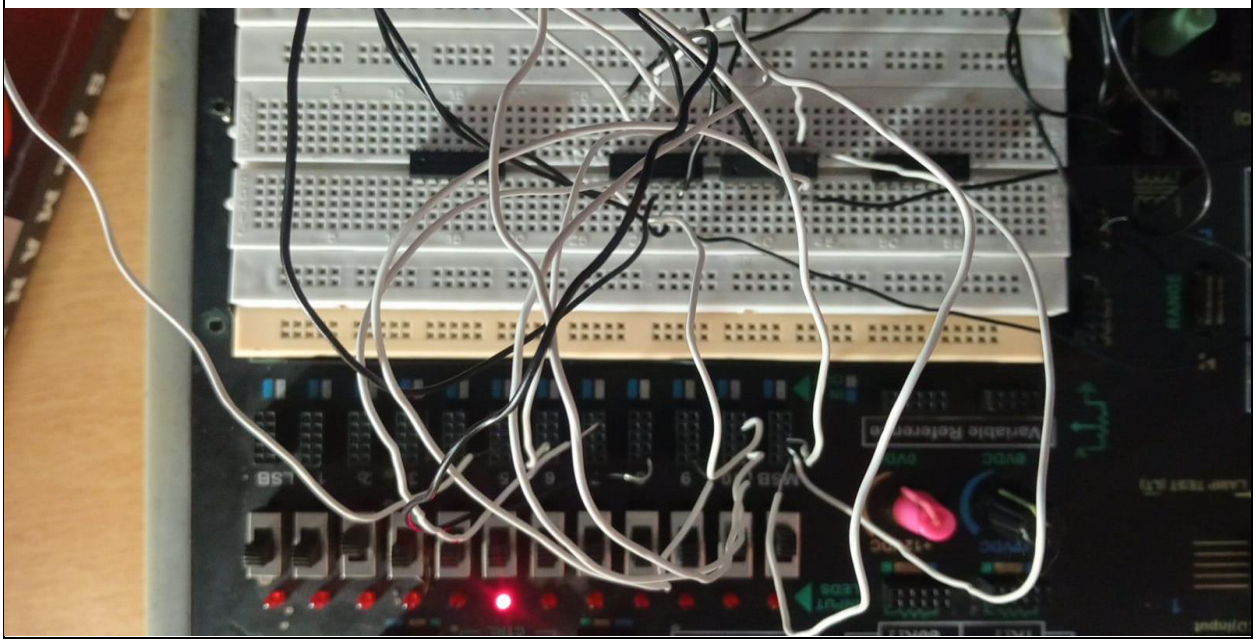
Input						Output
a	**b**	**A**	**B**	**C**	**D**	**Q**
0	**1**	**0**	**1**	**0**	**0**	**B(ON)**



Input						Output
a	b	A	B	C	D	Q
1	0	0	0	1	0	C(ON)



Input						Output
A	b	A	B	C	D	Q
1	1	0	0	0	1	D(ON)



Task 2:

Implement a 1x4 de-multiplexer. Write complete logic synthesis .

- **Single data input:** D
- **Select lines:** S1, S0
- **Outputs:** Y0, Y1, Y2, Y3
- The DEMUX routes the single input D to exactly one of the four outputs Y0..Y3 according to the select lines S1 S0. All other outputs are 0.

Truth Table:

Input			Output			
Y1	**Yo**	**F**	**A**	**B**	**C**	**D**
0	**0**	**0**	**0**	**0**	**0**	**0**
0	**0**	**1**	**1**	**0**	**0**	**0**
0	**1**	**0**	**0**	**0**	**0**	**0**
0	**1**	**1**	**0**	**1**	**0**	**0**
1	**0**	**0**	**0**	**0**	**0**	**0**
1	**0**	**1**	**0**	**0**	**1**	**0**
1	**1**	**0**	**0**	**0**	**0**	**0**
1	**1**	**1**	**0**	**0**	**0**	**1**

Logic Equations:

Canonical (Sum-of-Products style) output expressions (Use S1' to denote NOT S1, S0' to denote NOT S0):

A = F * S1' * S0'
B = F * S1' * S0
C = F * S1 * S0'
D = F * S1 * S0

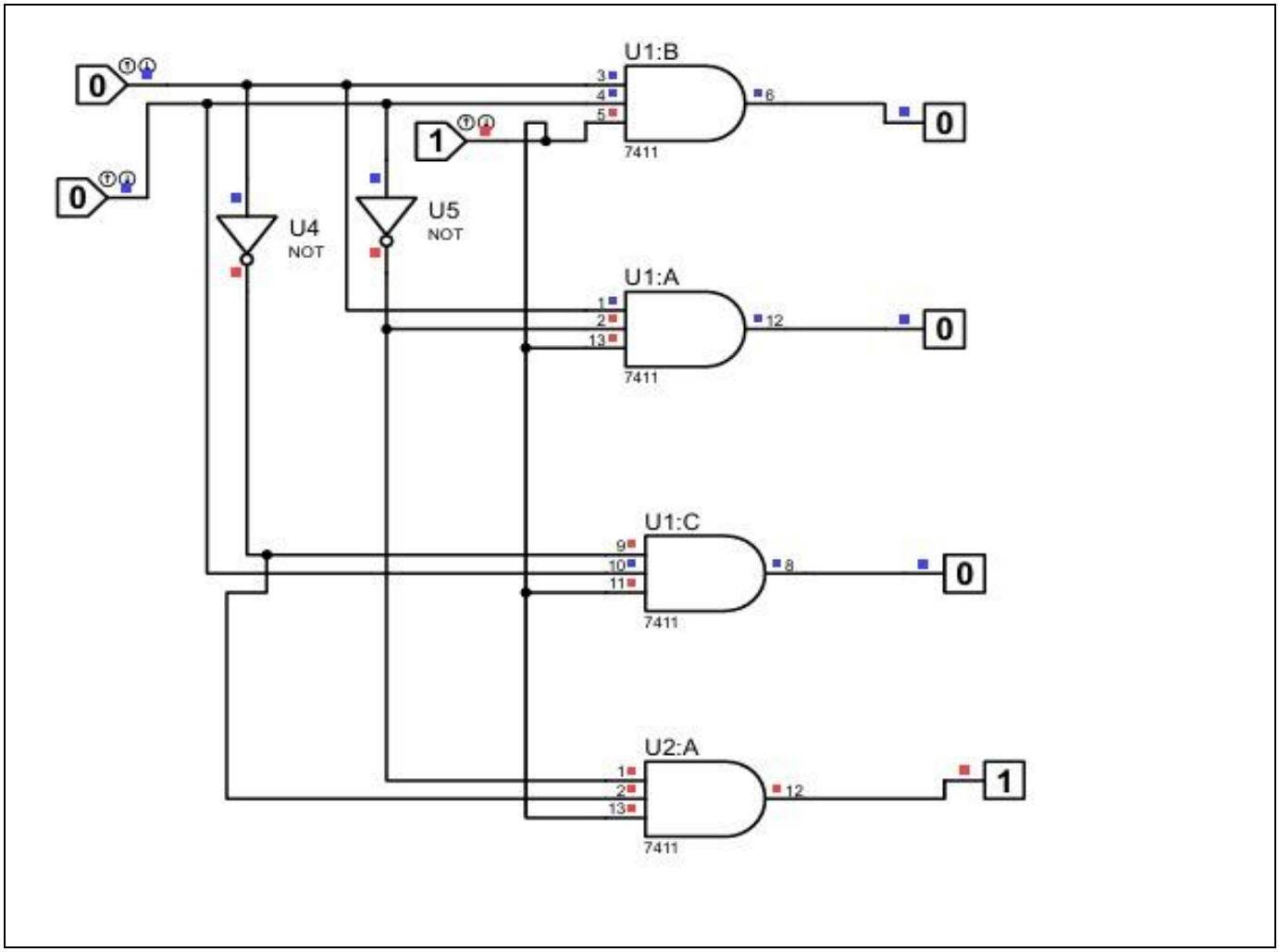
Logic-gate interpretation (intermediate terms T0..T3)
T0 = S1' * S0' (select pattern for A)
T1 = S1' * S0 (select pattern for B)
T2 = S1 * S0' (select pattern for C)
T3 = S1 * S0 (select pattern for D)

Then apply D as an enable:
A= F* T0
B = F * T1
C = F * T2
D = F * T3

Required logic gates

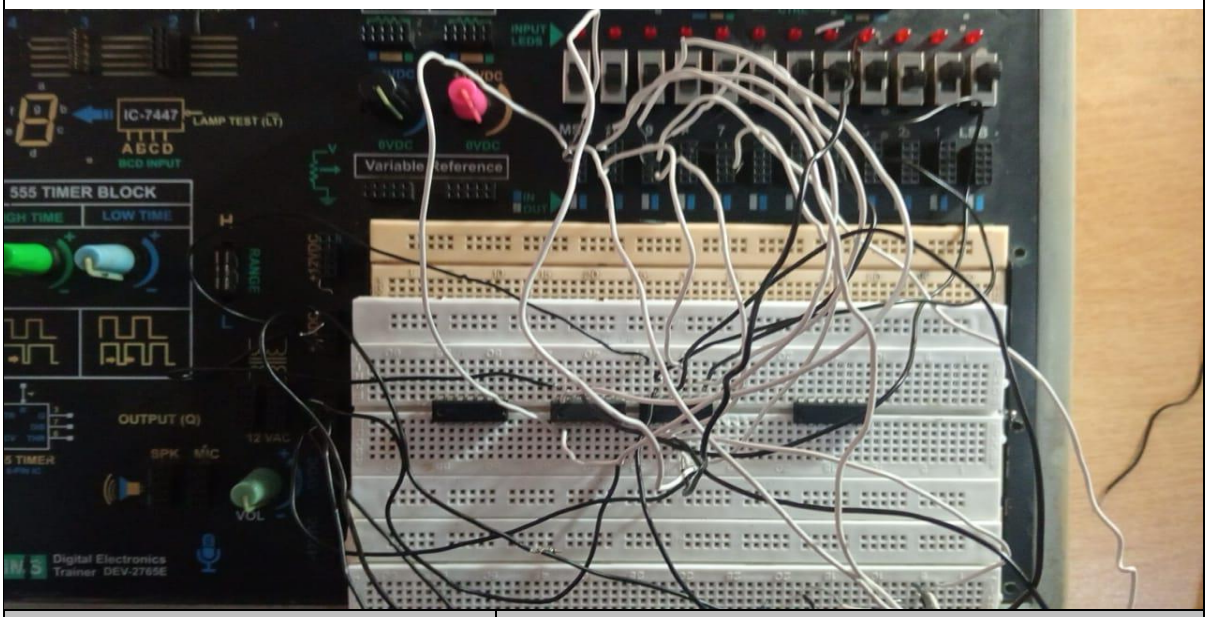
- **NOT gates:**
  - 1 NOT to produce S1' from S1
  - 1 NOT to produce S0' from S0
- **AND gates:**
  - Two 2-input ANDs to create selection terms (or use four 2-input ANDs directly)
  - Four 3-input AND gates (simpler): each takes D and the two select terms to produce Y0..Y3
  - Alternatively: build T0..T3 with 2-input ANDs, then 2-input AND with D to form each Yi

Logic Diagram:

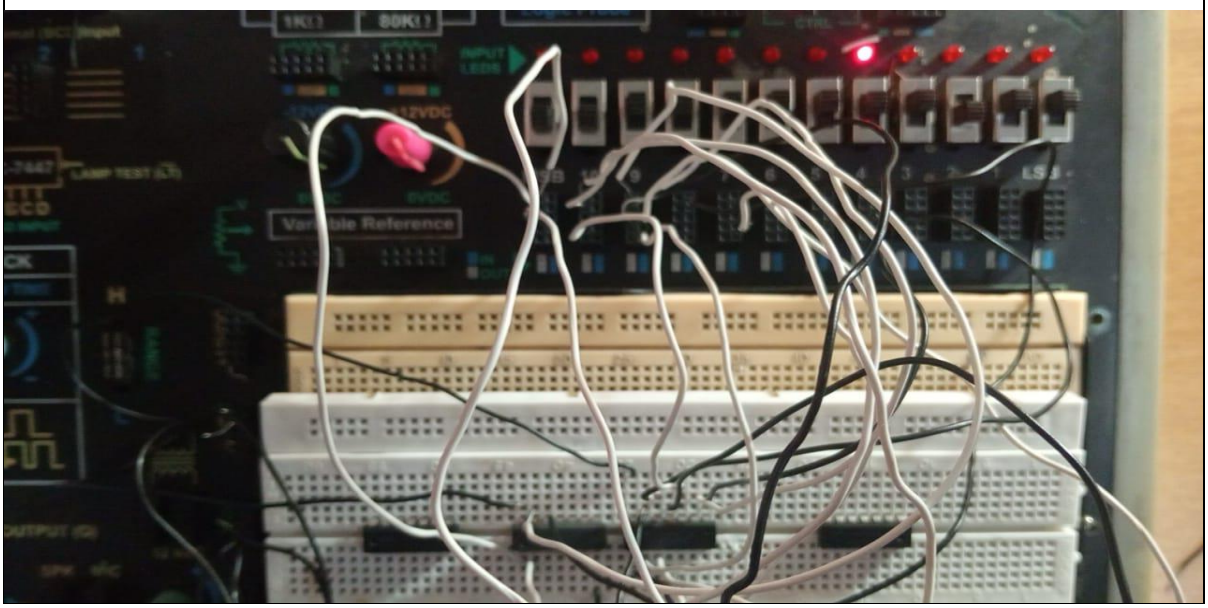


SIMULATION ACTIVITY:

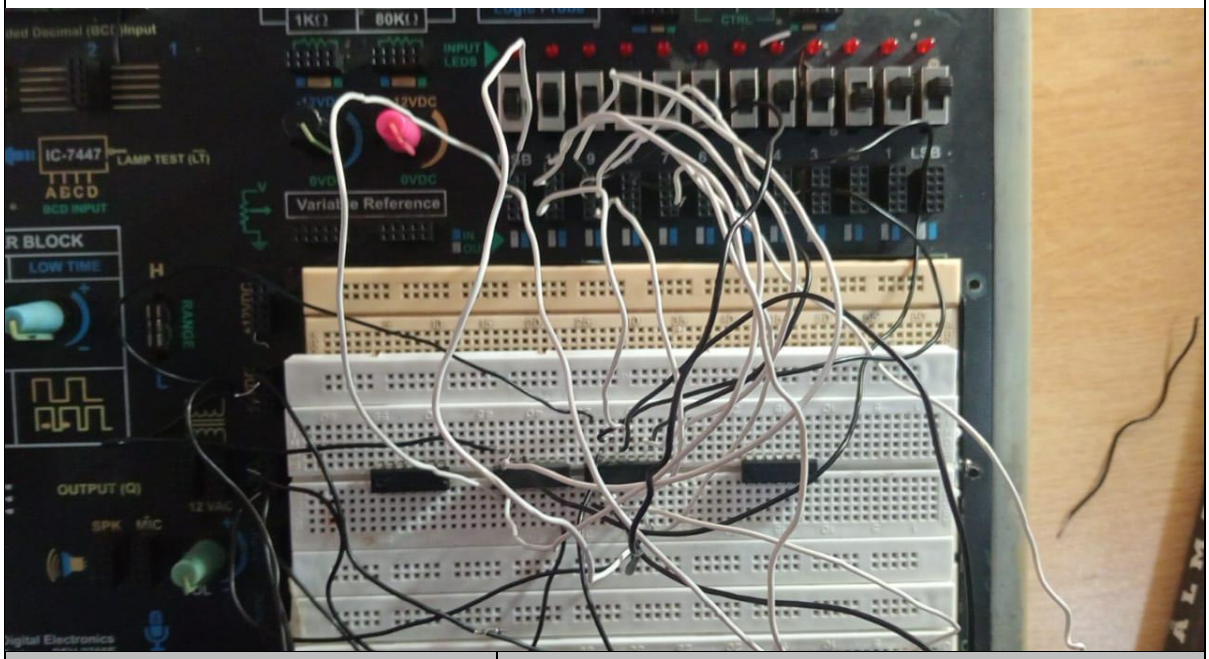
Input			Output			
Y1	**Yo**	**F**	**A**	**B**	**C**	**D**
0	**0**	**0**	**0**	**0**	**0**	**0**



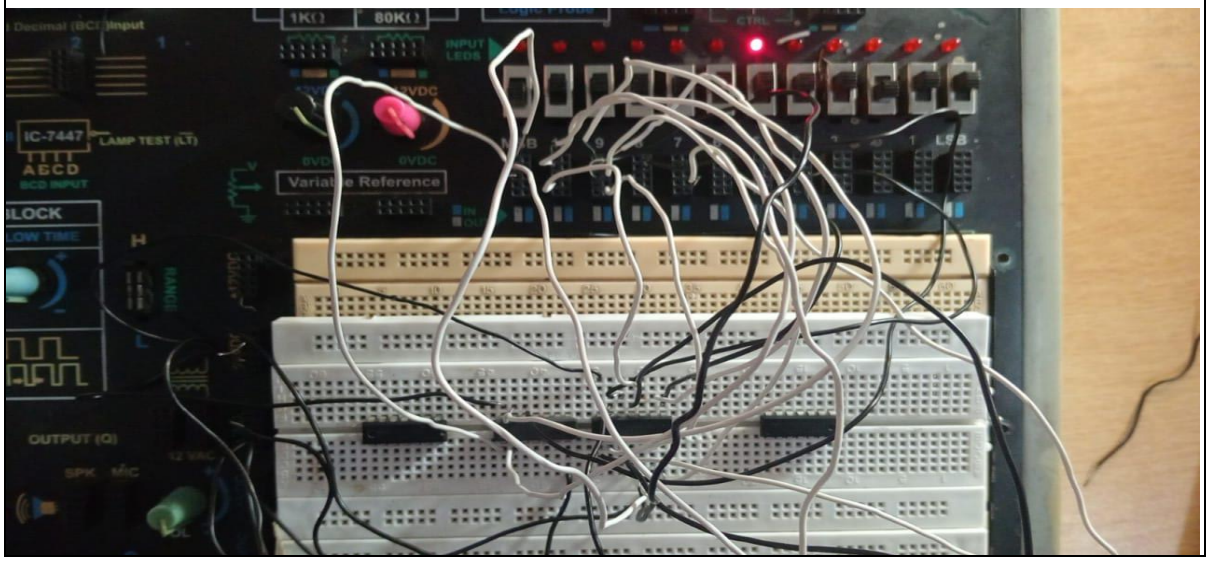
Input			Output			
A	**B**	**F**	**A**	**B**	**C**	**D**
0	**0**	**1**	**1**	**0**	**0**	**0**



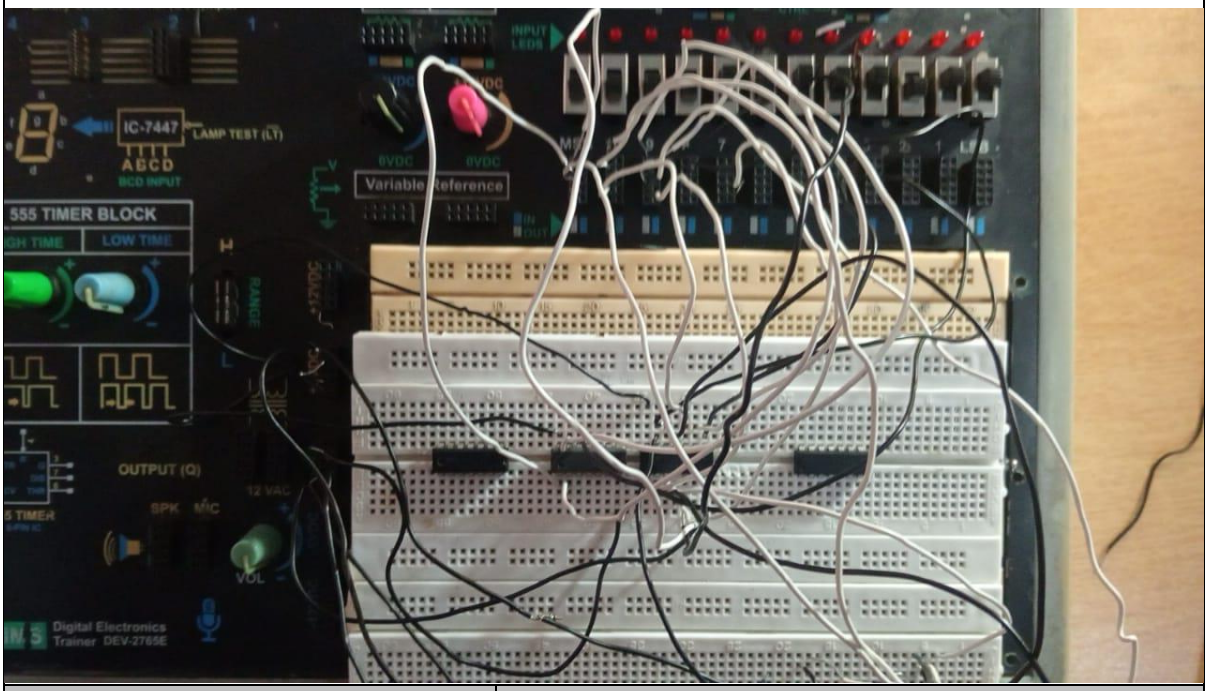
Input			Output			
Y1	Yo	F	A	B	C	D
0	1	0	0	0	0	0



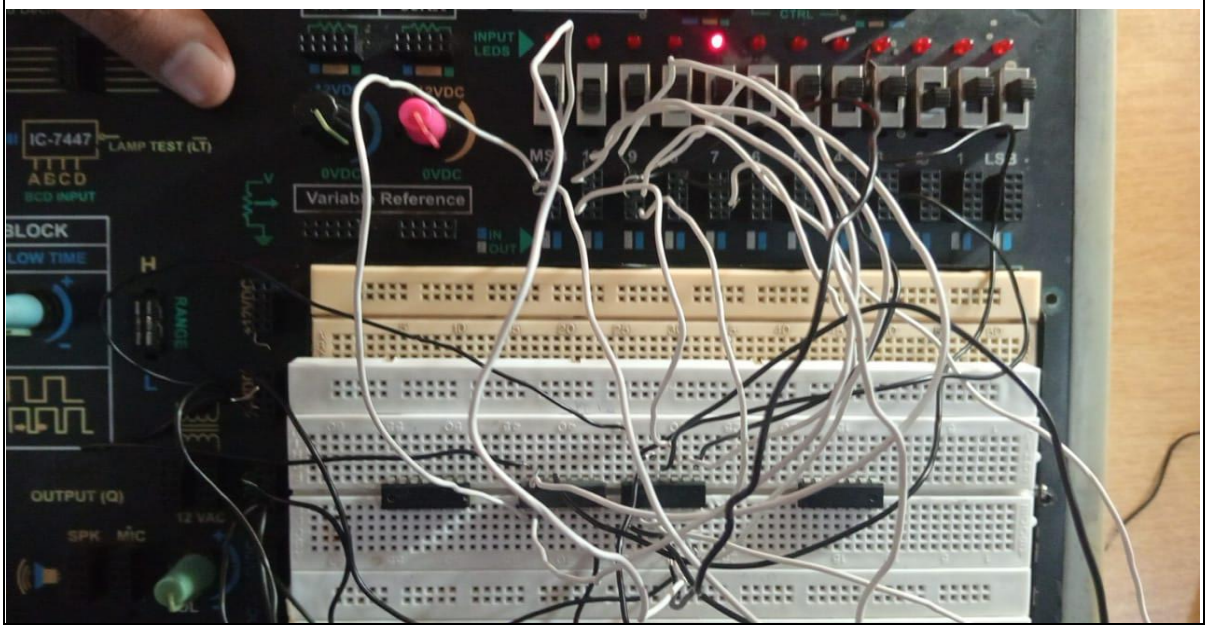
Input			Output			
A	B	F	A	B	C	D
0	1	1	0		0	0



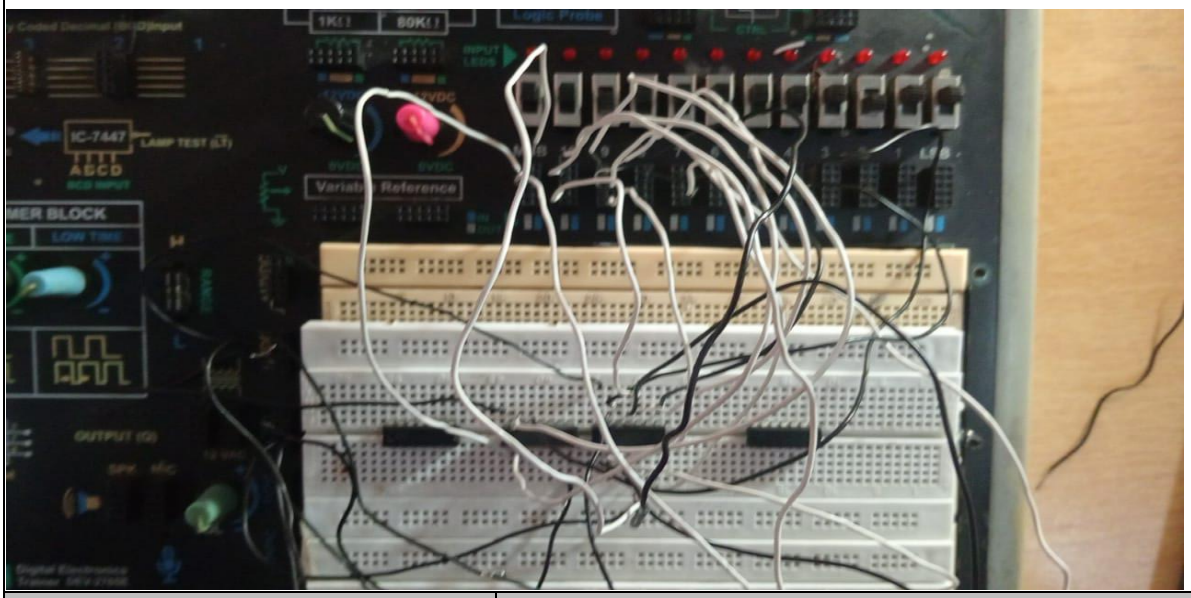
Input			Output			
Y1	Yo	F	A	B	C	D
1	0	0	0	0	0	0



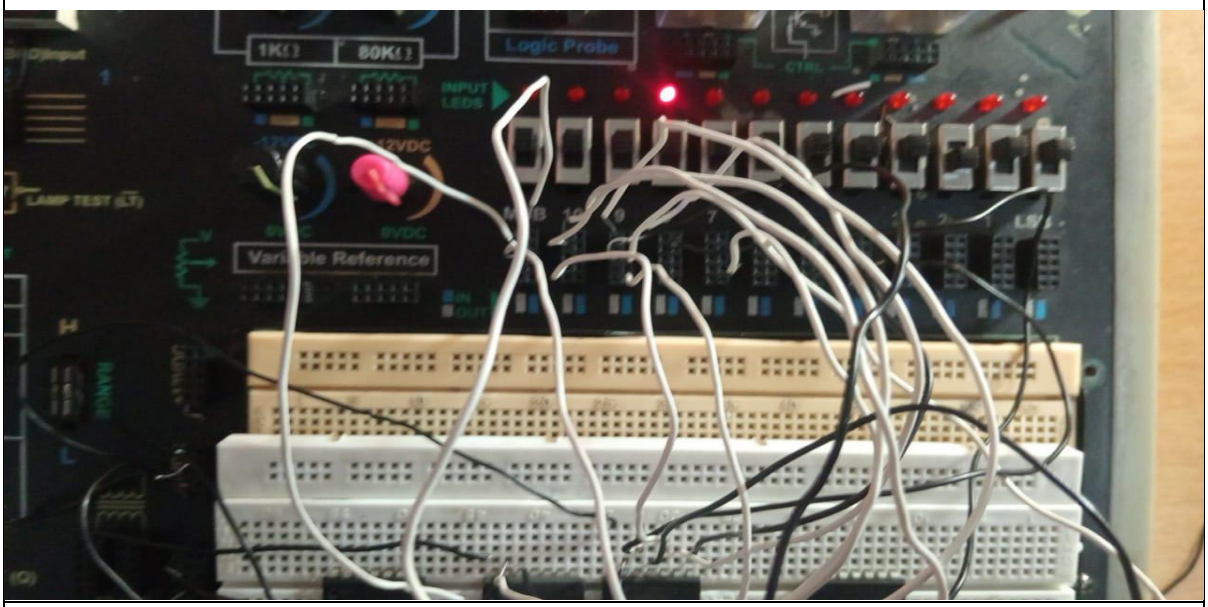
Input			Output			
A	B	F	A	B	C	D
1	0	1	0	0	1	0



Input			Output			
Y1	**Yo**	**F**	**A**	**B**	**C**	**D**
1	**1**	**0**	**0**	**0**	**0**	**0**



Input			Output			
A	**B**	**F**	**A**	**B**	**C**	**D**
1	**1**	**1**	**0**	**0**	**0**	**1**



Conclusion:

After designing and implementing the 4×1 multiplexer (MUX) and 1×4 demultiplexer (DEMUX) practically, it is observed that both circuits work exactly as expected. The MUX successfully selects one of the four inputs and routes it to the single output based on the two select lines, showing the principle of data selection clearly. Similarly, the DEMUX takes a single input and correctly directs it to one of the four outputs according to the select lines, while all other outputs remain zero, demonstrating data distribution. This practical exercise helps in understanding how digital circuits can control and manage data efficiently using select lines. Overall, these designs confirm that MUX and DEMUX are fundamental building blocks in digital electronics, useful for applications like communication systems, data routing, and signal processing.